

Lab 2: Fundamental Programming Concepts: Data Types, Variables, Operators, I/O and Type Casting

2.1 Objectives

By the end of this lab, students will be able to:

- Use cout and cin for console input and output.
- Declare and initialise variables and constants of a suitable data type.
- Apply arithmetic, assignment, increment, decrement, relational, logical and ternary operators.
- Understand operator precedence in expressions.
- Perform implicit and explicit type conversion (casting).
- Combine these concepts to solve simple real world calculations.

2.2 Software

Microsoft Visual Studio Community 2022

2.3 Theory

Every topic below has a small program, a plain-English explanation of what happens, and the output. Type each example in Visual Studio, run it, and compare with the output shown.

2.3.1 Output Construct (cout)

Console output in C++ is handled by the cout object from the <iostream> library. Data is sent to the screen using the insertion operator <<. Text, variables and expressions can be joined in one statement.

Example 1

```
#include <iostream>
using namespace std;

int main()
{
    int age = 20;

    cout << "Name: Ali" << endl;
    cout << "Age: " << age << endl;
    cout << "Age after 5 years: " << age + 5 << endl;
    return 0;
}
```

Line	What happens
#include <iostream>	Brings in the tools for input and output.
int age = 20;	Makes a box in memory named age and puts 20 inside it.
cout << "Age: " << age	Prints the text exactly as written, then prints the value stored in age.
age + 5	C++ calculates 20 + 5 = 25 first, then prints 25.

Output:

```
Name: Ali
Age: 20
Age after 5 years: 25
```

endl moves to a new line and flushes the output buffer. '\n' also moves to a new line but does not flush, so it is faster inside loops. Text inside double quotes is printed as it is; a variable name without quotes prints its value.

2.3.2 Input Construct (cin)

The cin object reads input from the keyboard using the extraction operator >>. The program waits until the user types a value and presses Enter.

Example 2

```
int number;

cout << "Enter a number: ";
cin >> number;
cout << "You entered: " << number << endl;
```

How it works: 1) The program prints the message. 2) It stops and waits. 3) The user types 7 and presses Enter. 4) The value 7 goes into the box number. 5) The program prints You entered: 7.

More than one value can be read in a single statement. The values are separated by a space or Enter:

Example 3

```
int length, width;

cout << "Enter length and width: ";
cin >> length >> width;
cout << "Area = " << length * width << endl;
```

How it works: If the user types 5 3 then length = 5 and width = 3, so the program prints Area = 15.

2.3.3 Variables

A variable is a named memory location that stores a value. The value can change while the program runs. Every variable must be declared with a data type before it is used.

Concept	Example
Declaration only	int age;
Declaration with initialisation	int age = 20;
Assignment after declaration	age = 25;
Multiple variables in one line	int x = 1, y = 2, z = 3;

Example 4: a variable can change

```
int score = 10;
cout << score << endl;   // prints 10
score = 50;              // old value 10 is replaced by 50
cout << score << endl;   // prints 50
```

Rules for naming variables: a name must start with a letter or underscore, may contain only letters, digits and underscores, cannot contain spaces, and cannot be a keyword. Names are case sensitive, so marks and Marks are different.

Valid Names	Invalid Names	Reason
totalMarks	2ndValue	Starts with a digit

_count	total marks	Contains a space
student1	float	float is a keyword
net_salary	net-salary	Hyphen is not allowed

2.3.4 Fundamental Data Types

A data type tells the compiler how much memory to reserve for a variable and what kind of value it can hold.

Data Type	Size (bytes)	Range / Use
char	1	-128 to 127, stores a single character
bool	1	true or false
short	2	-32,768 to 32,767
int	4	-2,147,483,648 to 2,147,483,647
float	4	About 7 significant decimal digits
double	8	About 15 significant decimal digits
string	Varies	Text such as "Ali" (needs the <string> header)

Example 5

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string name   = "Ali";       // text in double quotes
    int    age    = 21;          // whole number
    double gpa    = 3.75;        // decimal number
    char   grade  = 'A';         // one character in single quotes
    bool   passed = true;        // true or false

    cout << name << " is " << age << " years old." << endl;
    cout << "GPA: " << gpa << ", Grade: " << grade << endl;
    cout << "Passed (1=yes, 0=no): " << passed << endl;
    return 0;
}
```

Output:

```
Ali is 21 years old.
GPA: 3.75, Grade: A
Passed (1=yes, 0=no): 1
```

How it works: cout prints true as 1 and false as 0. Use 5.9f to make a float value; decimals are double by default.

The sizeof operator returns the number of bytes used by a data type or variable:

```
cout << "Size of int: "    << sizeof(int)    << endl;   // 4
cout << "Size of double: " << sizeof(double) << endl;   // 8
cout << "Size of char: "   << sizeof(char)   << endl;   // 1
```

2.3.5 Constants

A constant is a variable whose value cannot be changed after it is initialised. It is declared with the const keyword. By convention, constant names are written in capital letters.

Example 6

```
const double PI = 3.14159;
double radius = 2.5;

cout << "Area = " << PI * radius * radius << endl;

// PI = 3.14;   // ERROR: a constant cannot be modified
```

How it works: PI is locked. The area is 3.14159 x 2.5 x 2.5 = 19.6349. Trying to change PI gives a compiler error.

2.3.6 Operators Overview

An operator is a symbol that performs an operation on one or more values called operands. Based on the number of operands, operators are unary (one operand), binary (two operands) or ternary (three operands).

Category	Operator Type	Operators
Unary	Increment / Decrement	++ --
Binary	Arithmetic	+ - * / %
Binary	Assignment	= += -= *= /= %=
Binary	Relational	< <= > >= == !=
Binary	Logical	&& || (! is unary)
Ternary	Conditional	? :

2.3.7 Arithmetic Operators

Arithmetic operators perform mathematical calculations. When both operands are integers, division gives an integer result and the decimal part is dropped. The modulus operator % works only with integers; for decimal numbers use fmod() from <cmath>.

Operator	Meaning	Example (a = 10, b = 3)
+	Addition	a + b = 13
-	Subtraction	a - b = 7
*	Multiplication	a * b = 30
/	Division	a / b = 3 (decimal part dropped)
%	Modulus (remainder)	a % b = 1

Example 7

```
cout << 10 / 3          << endl;   // 3        integer division
cout << 10.0 / 3        << endl;   // 3.33333
cout << 10 % 3          << endl;   // 1
cout << fmod(10.5, 3.0) << endl;   // 1.5      needs <cmath>
```

How it works: 10 / 3: both are int, so the answer is int (3). 10 % 3: 10 divided by 3 is 3 remainder 1, so % gives 1. Making one side decimal (10.0 / 3) gives a decimal answer. Use n % 2 to test even (0) or odd (1).

2.3.8 Assignment Operators

The assignment operator = stores a value in a variable. Compound assignment operators combine an arithmetic operation with assignment to make the code shorter.

Operator	Example	Same As	Result (x = 10)
=	x = 10	x = 10	10
+=	x += 5	x = x + 5	15
-=	x -= 5	x = x - 5	5
*=	x *= 5	x = x * 5	50
/=	x /= 5	x = x / 5	2
%=	x %= 3	x = x % 3	1

Example 8

```
int total = 100;

total += 50;    // total is now 150
total -= 30;    // total is now 120
total /= 4;     // total is now 30
cout << total;  // prints 30
```

Statement	total becomes
int total = 100;	100
total += 50;	150
total -= 30;	120
total /= 4;	30

2.3.9 Increment and Decrement Operators

++ increases a variable by 1 and -- decreases it by 1. In prefix form (++a) the value is changed first and then used. In postfix form (a++) the old value is used first and then changed.

Example 9

```
int a = 5;

cout << ++a << endl;   // 6   (increment first, then print)
cout << a++ << endl;   // 6   (print first, then increment)
cout << a   << endl;   // 7
cout << a-- << endl;   // 7   (print first, then decrement)
cout << --a << endl;   // 5   (decrement first, then print)
```

Statement	Printed	a after
int a = 5;	-	5
++a	6	6
a++	6	7
a	7	7
a--	7	6

--a	5	5

How it works: Memory trick: sign BEFORE the variable = act before using. Sign AFTER the variable = act after using.

2.3.10 Relational Operators

Relational operators compare two values. The result is 1 (true) or 0 (false). They are mostly used in conditions and loops.

Operator	Meaning	Example (x = 5, y = 3)	Result
==	Equal to	x == y	0
!=	Not equal to	x != y	1
>	Greater than	x > y	1
<	Less than	x < y	0
>=	Greater than or equal to	x >= 5	1
<=	Less than or equal to	x <= 3	0

Example 10

```
int x = 5, y = 3;

cout << (x > y)  << endl;   // 1   (5 is greater than 3)
cout << (x == y) << endl;   // 0   (5 is not equal to 3)
```

Note: = is assignment and == is comparison. Writing x = y instead of x == y is a very common mistake.

2.3.11 Logical Operators

Logical operators combine two or more conditions into one.

Operator	Name	Rule
&&	Logical AND	True only if both conditions are true.
||	Logical OR	True if at least one condition is true.
!	Logical NOT	Reverses the result: true becomes false and false becomes true.

Example 11

```
int marks = 75, attendance = 80;

cout << (marks >= 50 && attendance >= 75) << endl;   // 1  both true
cout << (marks >= 90 || attendance >= 75) << endl;   // 1  one is true
cout << !(marks >= 50)                    << endl;   // 0  NOT of true
```

How it works: Line 1: 75 >= 50 is true AND 80 >= 75 is true, so the result is 1. Line 2: 75 >= 90 is false OR 80 >= 75 is true, so the result is 1. Line 3: 75 >= 50 is true, NOT true is 0.

2.3.12 Ternary Operator

The ternary operator is a short form of a simple if-else. It has three parts. Syntax:

```
variable = (condition) ? value_if_true : value_if_false;
```

Example 12

```
int a = 2, b = 7;
int max = (a > b) ? a : b;    // picks the larger value
cout << max;                  // prints 7
```

How it works: Is 2 > 7? No, so C++ takes the part after the colon, which is b = 7.

Example 13

```
int marks = 62;
string result = (marks >= 50) ? "Pass" : "Fail";
cout << result;               // prints Pass
```

Ternary operators can also be nested to check more than one condition:

Example 14

```
int num = -4;

cout << ((num > 0) ? "Positive" : (num < 0) ? "Negative" : "Zero");
// prints Negative
```

How it works: Is -4 > 0? No. Then check: is -4 < 0? Yes, so the answer is Negative.

2.3.13 Type Conversion (Casting)

Implicit Conversion

The compiler automatically converts a smaller data type to a larger one when they are used together, so no data is lost.

Example 15

```
int    count = 9;
double value = count;         // int converted to double automatically
cout << value + 0.5;          // prints 9.5
```

Explicit Conversion

The programmer forces a conversion using the cast operator. Syntax: (type) expression or static_cast<type>(expression).

Example 16

```
int total = 7, count = 2;

cout << total / count                          << endl;   // 3    integer division
cout << (float)total / count                   << endl;   // 3.5  after casting
cout << static_cast<double>(total) / count     << endl;   // 3.5  C++ style cast

float price = 99.99f;
int   whole = (int)price;                                 // 99   decimal part lost
cout << whole << endl;
```

How it works: (float)total turns 7 into 7.0 BEFORE the division, so the answer keeps its decimal part. Casting decimal to int simply cuts the decimal part (99.99 becomes 99, not 100).

Character and ASCII Conversion

Every character has a numeric ASCII code. For example A is 65, B is 66 and a is 97. Casting converts between the two:

Example 17

```
char ch = 'A';

cout << (int)ch        << endl;   // 65  character to ASCII
cout << (char)66       << endl;   // B   ASCII to character
cout << (char)(ch + 1) << endl;   // B   next character
```

How it works: 'A' + 1 = 65 + 1 = 66, and (char)66 is 'B'.

2.3.14 Operator Precedence

When an expression has more than one operator, C++ evaluates them according to precedence. Operators with the same precedence are evaluated from left to right. Parentheses always have the highest priority.

Precedence (High to Low)	Operators
1. Parentheses	( )
2. Unary	++ -- ! (sign) + -
3. Multiplicative	* / %
4. Additive	+ -
5. Shift	<< >>
6. Relational	< <= > >=
7. Equality	== !=
8. Logical AND	&&
9. Logical OR	||
10. Assignment	= += -= *= /= %=

Example 18: step by step evaluation

```
int r = 10 + 4 * 3 / 2 - 1;

// Step 1:  4 * 3       = 12
// Step 2:  12 / 2      = 6
// Step 3:  10 + 6      = 16
// Step 4:  16 - 1      = 15

int s = (10 + 4) * 3;   // parentheses first: 14 * 3 = 42
```

2.3.15 Putting it Together: Student Marks Calculator

The following program combines variables, constants, input, arithmetic operators and casting to calculate the total, average and percentage of a student.

Example 19

```
#include <iostream>
#include <string>
using namespace std;

int main()
```

```
{
    const int TOTAL_MARKS = 300;
    string name;
    int pf, calculus, ict;

    cout << "Enter student name: ";
    cin >> name;
    cout << "Enter marks of PF, Calculus and ICT: ";
    cin >> pf >> calculus >> ict;

    int total = pf + calculus + ict;
    double average = static_cast<double>(total) / 3;
    double percentage = static_cast<double>(total) / TOTAL_MARKS * 100;

    cout << "Name: " << name << endl;
    cout << "Total Marks = " << total << endl;
    cout << "Average Marks = " << average << endl;
    cout << "Percentage = " << percentage << "%" << endl;
    return 0;
}
```

Output (for marks 78, 84 and 91):

```
Name: Ali
Total Marks = 253
Average Marks = 84.3333
Percentage = 84.3333%
```

How it works: total = 78 + 84 + 91 = 253. average = 253.0 / 3 = 84.3333. percentage = 253.0 / 300 x 100 = 84.3333. Without static_cast<double>, 253 / 3 would be integer division and give 84.

2.4 In-Class Activity (Perform in Lab)

For Activities 1 and 2, first predict the output on paper, then type and run the code to check your answer. Show your instructor when done.

Activity 1: Predict the output

```
int a = 4, b = 9;
cout << a + b * 2   << endl;
cout << (a + b) * 2 << endl;
cout << b / a       << endl;
cout << b % a       << endl;
cout << (double)b / a << endl;
```

Line	Your prediction	Actual output
1		
2		
3		
4		
5		

Activity 2: Increment trace

```
int n = 10;
cout << n++ << endl;
cout << ++n << endl;
cout << n-- << endl;
cout << n   << endl;
```

Statement	Printed	n after
n++		
++n		
n--		
n		

Activity 3: Rectangle

Take the length and width from the user. Display the area and the perimeter. Hint: area = l x w, perimeter = 2 x (l + w).

Activity 4: Celsius to Fahrenheit

Take a temperature in Celsius (double) and display it in Fahrenheit. Hint: F = C * 9 / 5 + 32. Test with 37, expected 98.6.

Activity 5: Even or Odd

Take an integer and display Even or Odd using the % operator and the ternary operator.

Activity 6: Character magic

Take one character from the user. Display its ASCII code and the next character. Test with 'a', expected 97 and b.

Activity 7: Debug hunt

Find and fix the 4 errors in this program:

```
#include <iostream>
using namespace std;
int main()
{
    int 2ndNumber = 5;
    float total marks = 40;
    const int LIMIT = 10;
    LIMIT = 20;
    cout << "Value: " << 2ndNumber
    return 0;
}
```

2.5 Home Task

Complete all tasks and include screenshots of your code and output in the lab report.

1. Write a program that takes two integers from the user and displays their sum, difference, product, quotient and remainder. Choose suitable variables and use the required arithmetic operators.

2. Write a program to calculate the percentage of a student who obtains 425 marks out of 500. Store the marks as integers and use explicit type casting to get an accurate decimal result. The expected percentage is 85%.

3. Write a currency converter that converts Rs. 12,500 into another currency using the exchange rate given by the instructor (see the table below). Store the exchange rate in a double and display the original amount, the exchange rate and the converted amount.

Conversion	Given Amount	Exchange Rate	Formula
PKR to USD	Rs. 12,500	1 USD = Rs. 280	12500 / 280
PKR to EUR	Rs. 12,500	1 EUR = Rs. 330	12500 / 330
PKR to GBP	Rs. 12,500	1 GBP = Rs. 380	12500 / 380
PKR to AED	Rs. 12,500	1 AED = Rs. 76	12500 / 76
PKR to SAR	Rs. 12,500	1 SAR = Rs. 75	12500 / 75

4. Write a program for a parking counter. The parking area starts with 20 available spaces. When a vehicle enters, the available spaces decrease by one, and when a vehicle leaves, they increase by one. Use increment and decrement operators to update the spaces, and show the difference between the prefix and postfix forms.

5. Write a program that takes two integers from the user and demonstrates binary operators. Perform arithmetic, relational and logical operations on the values and display the result of each operation with a clear label.

6. Write a program that takes the obtained marks of a student and displays Pass if the marks are 50 or above, otherwise Fail. Use the ternary operator instead of an if-else statement.

7. Mini Application: Student Semester Fee Calculator. Take the student's name, registration number and number of registered credit hours as input. Store the fee per credit hour as a constant, calculate the tuition fee, add a fixed registration or service charge and calculate the final semester fee. Display the result as a simple Semester Fee Slip showing the student's information, credit hours, fee per credit hour, tuition fee, additional charges and total payable amount.

Hints

- Task 2: cast before dividing, static_cast<double>(obtained) / total * 100.
- Task 3: use 12500.0 / rate so the answer keeps its decimal part.
- Task 4: print spaces-- and --spaces on separate lines and compare the two results.
- Task 7: tuition = creditHours * FEE_PER_CREDIT; total = tuition + SERVICE_CHARGE.

2.6 Conclusion

Write a brief conclusion summarising what you learned in this lab:



Appendix: Solutions (Instructor Copy – remove before giving to students)

In-Class Activity Solutions

Activity 1 answers: 22, 26, 2, 1, 2.25. Activity 2 answers: 10, 12, 12, 11. Activity 7 errors: variable name starts with a digit; space in a variable name; a const cannot be changed; missing semicolon after the cout line.

```
// Activity 3: Rectangle
#include <iostream>
using namespace std;
int main()
{
    int l, w;
    cout << "Enter length and width: ";
    cin >> l >> w;
    cout << "Area = " << l * w << endl;
    cout << "Perimeter = " << 2 * (l + w) << endl;
    return 0;
}
```

```
// Activity 4: Celsius to Fahrenheit
#include <iostream>
using namespace std;
int main()
{
    double c;
    cout << "Enter temperature in Celsius: ";
    cin >> c;
    double f = c * 9 / 5 + 32;
    cout << "Fahrenheit = " << f << endl;
    return 0;
}
```

```
// Activity 5: Even or Odd
#include <iostream>
using namespace std;
int main()
{
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << ((n % 2 == 0) ? "Even" : "Odd") << endl;
    return 0;
}
```

```
// Activity 6: Character magic
#include <iostream>
using namespace std;
int main()
{
    char ch;
    cout << "Enter a character: ";
```

```
    cin >> ch;
    cout << "ASCII code = " << (int)ch << endl;
    cout << "Next character = " << (char)(ch + 1) << endl;
    return 0;
}
```

Home Task Solutions

```
// Task 1
#include <iostream>
using namespace std;
int main()
{
    int a, b;
    cout << "Enter two integers: ";
    cin >> a >> b;
    cout << "Sum = " << a + b << endl;
    cout << "Difference = " << a - b << endl;
    cout << "Product = " << a * b << endl;
    cout << "Quotient = " << a / b << endl;
    cout << "Remainder = " << a % b << endl;
    return 0;
}
```

```
// Task 2
#include <iostream>
using namespace std;
int main()
{
    int obtained = 425, total = 500;
    double percentage = static_cast<double>(obtained) / total * 100;
    cout << "Percentage = " << percentage << "%" << endl;   // 85%
    return 0;
}
```

```
// Task 3: currency converter (all five currencies)
#include <iostream>
using namespace std;
int main()
{
    const double AMOUNT = 12500;
    double usd = 280, eur = 330, gbp = 380, aed = 76, sar = 75;

    cout << "Original amount: Rs. " << AMOUNT << endl;
    cout << "USD rate " << usd << " -> " << AMOUNT / usd << " USD" << endl;
    cout << "EUR rate " << eur << " -> " << AMOUNT / eur << " EUR" << endl;
    cout << "GBP rate " << gbp << " -> " << AMOUNT / gbp << " GBP" << endl;
    cout << "AED rate " << aed << " -> " << AMOUNT / aed << " AED" << endl;
    cout << "SAR rate " << sar << " -> " << AMOUNT / sar << " SAR" << endl;
    return 0;
}
```

Expected: USD 44.6429, EUR 37.8788, GBP 32.8947, AED 164.474, SAR 166.667.

```
// Task 4: parking counter
```

```
#include <iostream>
using namespace std;
int main()
{
    int spaces = 20;
    cout << "Available spaces: " << spaces << endl;                 // 20
    cout << "Vehicle enters (postfix): " << spaces-- << endl;      // prints 20, then 19
    cout << "Now available: " << spaces << endl;                    // 19
    cout << "Vehicle enters (prefix): " << --spaces << endl;       // 18
    cout << "Vehicle leaves (prefix): " << ++spaces << endl;       // 19
    cout << "Vehicle leaves (postfix): " << spaces++ << endl;      // prints 19, then 20
    cout << "Final available: " << spaces << endl;                  // 20
    return 0;
}
```

Key point: postfix prints the OLD value, prefix prints the NEW value.

```
// Task 5: binary operators
#include <iostream>
using namespace std;
int main()
{
    int a, b;
    cout << "Enter two integers: ";
    cin >> a >> b;
    cout << "a + b = " << a + b << endl;
    cout << "a - b = " << a - b << endl;
    cout << "a * b = " << a * b << endl;
    cout << "a / b = " << a / b << endl;
    cout << "a % b = " << a % b << endl;
    cout << "a > b  : " << (a > b)  << endl;
    cout << "a < b  : " << (a < b)  << endl;
    cout << "a == b : " << (a == b) << endl;
    cout << "a != b : " << (a != b) << endl;
    cout << "a > 0 && b > 0 : " << (a > 0 && b > 0) << endl;
    cout << "a > 0 || b > 0 : " << (a > 0 || b > 0) << endl;
    cout << "!(a > b)       : " << !(a > b) << endl;
    return 0;
}
```

```
// Task 6: Pass / Fail
#include <iostream>
using namespace std;
int main()
{
    int marks;
    cout << "Enter obtained marks: ";
    cin >> marks;
    cout << ((marks >= 50) ? "Pass" : "Fail") << endl;
    return 0;
}
```

```
// Task 7: Semester Fee Calculator
#include <iostream>
#include <string>
using namespace std;
int main()
```

```
{
    const int FEE_PER_CREDIT = 8000;     // change to the instructor's value
    const int SERVICE_CHARGE = 5000;     // change to the instructor's value

    string name, regNo;
    int creditHours;

    cout << "Enter student name: ";
    cin >> name;
    cout << "Enter registration number: ";
    cin >> regNo;
    cout << "Enter credit hours: ";
    cin >> creditHours;

    int tuition = creditHours * FEE_PER_CREDIT;
    int total = tuition + SERVICE_CHARGE;

    cout << "\n===== SEMESTER FEE SLIP =====" << endl;
    cout << "Name               : " << name << endl;
    cout << "Registration No    : " << regNo << endl;
    cout << "Credit Hours       : " << creditHours << endl;
    cout << "Fee per Credit     : Rs. " << FEE_PER_CREDIT << endl;
    cout << "Tuition Fee        : Rs. " << tuition << endl;
    cout << "Additional Charges : Rs. " << SERVICE_CHARGE << endl;
    cout << "Total Payable      : Rs. " << total << endl;
    return 0;
}
```

Sample: 15 credit hours gives tuition 120000 and total 125000.